

What is JavaScript?

JavaScript is a **programming language** used to make web pages **interactive, dynamic, and functional**.

- Runs in browser and server
- Supports multiple programming styles
- Used in frontend and backend

Why do we need JavaScript?

- To add **interactivity** (clicks, events)
- To create **dynamic content**
- To handle **user input**
- To communicate with **servers (APIs)**
- To build **full web applications**

1. Data Types - Data types in JavaScript define **what kind of value a variable can hold and how that value behaves in operations**.

JavaScript is a **dynamically typed language**, which means:

- You don't need to declare the type explicitly
- The type is decided automatically at runtime

Ex: let x = 10; // number

 x = "Hello"; // now string

Types of Data Types

JavaScript has **2 main categories**:

1. Primitive Data Types

These are **basic, simple, immutable values** (stored directly in memory)

There are **7 primitive types**:

(1) Number- Used to store **numeric values (both integers and decimals)**.

let a = 10;

let b = 3.14;

Special values:

- NaN → Not a number
- Infinity

```
console.log(0.1 + 0.2); // precision issue
```

(2) BigInt - Used for **very large numbers** beyond normal limit.

```
let big = 12345678901234567890n;
```

(3) String - Used to store **text (sequence of characters)**.

```
let name = "Sai";
```

- Strings are **immutable**

(4) Boolean- Used for **true/false values** (mainly in conditions).

```
let isValid = true;
```

(5) Undefined - A variable that is **declared but not assigned a value**.

```
let x;
```

```
console.log(x); // undefined
```

(6) Null - Represents an **intentional empty value**.

```
let data = null;
```

Difference:

- undefined → no value assigned
- null → value intentionally empty

(7) Symbol- Used to create **unique identifiers**.

```
let id = Symbol("id");
```

2. Non-Primitive (Reference) Data Types - These are **complex data structures** stored by reference.

(1) Object - Collection of **key-value pairs**

```
let user = {
```

```
name: "Sai",  
age: 19  
};
```

(2) Array - Used to store **multiple values in order**

```
let arr = [1, 2, 3];
```

(3) Function - Functions are also objects in JavaScript.

```
function greet() {  
    return "Hello";  
}
```

2. Variables - Variables are used to **store data values in memory** so that they can be used and modified later.

JavaScript provides **three ways to declare variables**:

Types of Variables

(1) var

- Old method (ES5)
- **Function scoped**
- Can be redeclared and updated

```
var x = 10;
```

```
var x = 20; // allowed
```

Problem: causes bugs due to scope issues

(2) let

- Introduced in ES6
- **Block scoped** (only inside { })
- Can be updated but not redeclared

```
let a = 10;
```

```
a = 20; // allowed
```

```
// let a = 30 ❌ error
```

(3) const

- Block scoped
- Cannot be reassigned
- Must be initialized

```
const b = 10;
```

```
// b = 20 ❌ error
```

👉 But objects/arrays inside const can change:

```
const arr = [1,2];
```

```
arr.push(3); // allowed
```

3. Operators - Operators are used to **perform operations on variables and values**.

Types of Operators

(1) Arithmetic Operators - + - * / % **

```
let a = 10 + 5; // 15
```

(2) Comparison Operators

```
== // loose equality
```

```
=== // strict equality
```

```
console.log(5 == "5"); // true
```

```
console.log(5 === "5"); // false
```

(3) Logical Operators

```
&& || !
```

```
console.log(true && false); // false
```

(4) Assignment Operators

`= += -=`

`let x = 5;`

`x += 2; // 7`

4. Control Structures - Control structures control the **flow of execution**.

Conditional Statements

`if (condition) {`

`}`

`else {`

`}`

Loops

for loop

`for (let i = 0; i < 3; i++) {`

`console.log(i);`

`}`

while loop

`while (x < 5) {`

`x++;`

`}`

do-while

`do {`

`x++;`

`} while (x < 5);`

Special Loops

```
for (let val of arr) {}
```

```
for (let key in obj) {}
```

5. Objects - Objects store data in **key-value pairs** and represent real-world entities.

♦ **Creating Object**

```
let user = {  
  name: "Sai",  
  age: 19  
};
```

♦ **Accessing Values**

```
user.name  
user["age"]
```

6. Arrays- Arrays store **multiple values in a single variable (ordered collection)**.

Creating Array

```
let arr = [1, 2, 3];
```

Operations

```
arr.push(4);  
arr.pop();
```

Important Points

- Dynamic size
- Can store mixed types

```
let arr = [1, "Hello", true];
```

7. Functions - Functions are **reusable blocks of code**.

Basic Function

```
function add(a, b) {  
  return a + b;  
}
```

8. Classes - Classes are **blueprints to create objects**.

Example

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

Features

- Constructor
- Methods
- Inheritance

10. Asynchronous Programming - JavaScript is **single-threaded but asynchronous**.

Types

Callback - is a function that is executed after an asynchronous task (like API call, timer, file read) completes.

```
setTimeout(() => {}, 1000);
```

Promise - is an object that represents the result of an asynchronous operation, which may be available now, later, or never.

```
fetch(url).then().catch();
```

11. Modules - Modules help organize code into **separate reusable files**.

Example

```
export const a = 10;  
  
import { a } from "./file.js";
```

12. Error Handling - Used to handle runtime errors safely.

Example

```
try {  
  
  let x = y;  
  
} catch (e) {  
  
  console.log(e);  
  
}
```

BEST PRACTICES

1. Block Scoped Declarations - A variable is accessible only inside the block { } where it is declared

- var → not block scoped (causes bugs)
- let, const → block scoped
- const → cannot be reassigned
- Fixes scope issues of var

2. Arrow Functions - An arrow function is a short and modern way to write functions in JavaScript.

- Introduced in ES6 to make code more concise and readable
- Short syntax: =>
- No own this
- Cleaner code
- Makes code concise and readable

3. Optional Chaining (?.) - Optional chaining (?.) is used to safely access nested object properties.

- If a value is null or undefined, it stops execution and returns undefined instead of throwing an error
- Safely access nested objects

- Avoids runtime errors
- Returns undefined instead of error

4. Nullish Coalescing (??) - The nullish coalescing operator (??) is used to provide a default value only when the variable is:

- **null**
- **Undefined**
- **It avoids replacing valid values like 0, false, or ""**
- Default value only for:
 - null
 - undefined
 - Unlike ||, keeps valid values like 0, ""

5. Logical Assignment - Logical assignment operators are shortcuts to assign values based on conditions.

- Short way to assign values

Types:

- ??= → if null/undefined
- ||= → if falsy
- &&= → if truthy
- Simplifies conditional logic

6. Named Capture Groups - A Named Capture Group is a feature in regular expressions (regex) that lets you capture parts of a string using names instead of numbers.

- Used in regex
- Access captured values using names instead of index
- Improves readability

7. Async & Await

async → makes a function return a Promise

await → pauses execution until Promise resolves

- Used for asynchronous programming
- Works with Promises

- Makes async code look synchronous
- Simplifies API handling